

Circadia Command - Multi-Tenant, Subscription and Data Isolation

Project outline and architecture review (v2)

Document date: 5 July 2026

Status: Architecture locked for planning - not yet implemented

Repository: `fatigue-app-latest` (`app-next` + `circadia-command`)

Executive summary

Circadia today runs as a single pilot customer on one shared Neon Postgres database. Command lifecycle tables already carry `tenant_id_uuid`, but the customer app has no Tenant/Organization model, operators see a global triage queue, and there is no subscription or billing layer.

This document combines the original project outline with a design review synopsis and codebase-aligned refinements for migrating to multi-tenant B2B SaaS: tenant assignment, Command operator desks, subscription entitlements, and database containerization.

Recommended path: Option A (shared DB + selective RLS) first, with a hybrid database routing service so enterprise customers can upgrade to Neon branches or projects without forking application code.

Part A - Current state

Layer | Reality today

|-----|-----|

Customer app (Manager/Driver) | No Tenant table. Users, drivers, sheets, settings are deployment-global.

Command | Incidents have `tenant_id_uuid` from env `COMMAND_PILOT_TENANT_ID_UUID`.

Command operators | Global - any operator sees the entire triage queue.

Manager Live alerts | Queue filtered to pilot tenant; Command desk is not tenant-filtered.

Neon RLS | `fatigue_incident_lifecycle` has global operator bypass, not per-customer.

Billing | Not implemented.

Key code paths:

- `app-next/src/lib/integrations/command-lifecycle-bridge.ts` - Autonomise to `edge_fatigue_events`
- `app-next/src/lib/integrations/triage-active-queue.ts` - manager queue scoped to pilot tenant
- `circadia-command/src/lib/triage-queue.ts` - operator queue global (no tenant filter)
- `circadia-command/src/lib/privileged-db.ts` - RLS session claims for operators

Related docs: `incident-routing-assembly.md` (M1-M4), `MASTER_SPEC.md`, `SCHEMA_ADAPTATIONS.md`

Part B - Target model

Three capabilities must exist together:

1. Tenant entity - paying fleet/customer (name, status, stable tenant_id_uuid, isolation tier).
2. Assignment - Command operators to tenant(s); managers and drivers to tenant.
3. Entitlement - subscription gates Command desk, live alerts (M1-M4), assurance-only, seat limits.

Customer-facing routing (one choice at onboarding)

Choice | Command role

|-----|-----|

M1 - Circadia desk, supervisor must approve | Operator triage -> manager validation

M2 - Circadia desk only | Operator triage -> auto close/intervene

M3 - Supervisor on phone | Skip Command; manager inbox only

M4 - Circadia desk -> customer SOC | Operator triage -> webhook to customer SOC

Assurance only | No Command queue; heatmap/coaching only

Operator assignment (new)

```
operator_tenant_assignments
  operator_id    -> command_operators
  tenant_id_uuid -> tenants
  assignment_tier INT DEFAULT 1 -- 1 = Primary, 2 = Overflow/Backup
  is_active      BOOLEAN DEFAULT true
  PRIMARY KEY (operator_id, tenant_id_uuid)
```

Command /trriage filters: tenant_id_uuid IN (assigned tenants for this operator), tier 1 first.

Part C - Database isolation options

Option A - Shared Neon + tenant_id + selective RLS (default for SMB)

One database; tenant_id_uuid on rows; Postgres RLS on operator/manager-visible tables.

Refinement: Do NOT apply RLS on high-frequency edge_fatigue_events ingest. Use service ingest path with explicit tenant_id_uuid. RLS on lifecycle, app-next tenant tables, operator reads.

Option B - Neon branch per customer (mid-market upgrade)

Tenant.databaseUrl resolved by routing service. Same apps, dynamic connection.

Option C - Separate Neon project (enterprise)

Full physical isolation per ADR 0002 cutover pattern.

Hybrid database routing service (design review - adopt)

Decouple process.env.DATABASE_URL from direct Prisma init. Internal router accepts tenant_id_uuid + isolation tier (shared | branch | project) and resolves connection string. Enables Option A now with seamless B/C upgrade.

Data portability (design review - adopt)

Composite keys (tenant_id_uuid, id) or partition by tenant_id_uuid on high-throughput event tables. Simplifies pg_dump extraction when SMB scales to dedicated container.

Part D - Ingest resiliency and driver identity

Problem

deterministicDriverUuid(tenant + name + rego) in command-lifecycle-bridge.ts is stable but not roster-grounded. Name/regio changes fork identity. Multi-tenant commercial rollout on heuristic hashes corrupts historical reporting.

Design review recommendations (adopt with refinements)

Asynchronous unmapped events catchment

- Do not block or drop Autonomise telemetry if identity sync has not processed a driver.
- Store driver_id_uuid as provisional hash OR mapped UUID from identity_uuid_map.
- Add identity_status on lifecycle/edge row: mapped | unmapped | provisional.

Manager reconciliation UI

- "Unassigned alerts" view in app-next /manager/alerts.
- Fleet managers retroactively map telemetry to roster profiles.
- Eliminates data loss from roster sync lag.

Identity anchor

- identity_uuid_map is source of truth (MASTER_SPEC section 7).
 - Deterministic hash is fallback only, flagged provisional.
-

Part E - Operator triage and queue cascading

Tier 1 (Phase 3 - required for multi-tenant pilot)

Scope Command queue and SSE by operator_tenant_assignments and tenant_id_uuid. Replace global fetchTriageQueue filter.

Tier 2 cascade (Phase 6 - defer)

If incident unaddressed beyond SLA, cascade to assignment_tier = 2 backup operators. Requires timers, audit log, SSE refresh, ops runbook. Do not block first multi-tenant pilot.

Part F - Routing state machine determinism

Requirement (adopt)

Codify M1-M4 inside Command/lifecycle transition code - not UI feature flags. Single routing-mode module consumed by bridge, Command mutate, manager triage.

tenant_compliance_policy_overrides table exists but is not wired in TS today - must be read on every transition.

Fail-safe on config failure (refined)

Do NOT silently downgrade M2 to M3 on entitlement timeout - that changes contracted behaviour.

Preferred behaviour:

1. Cached tenant policy with TTL and last-known-good.
2. On failure: ingest continues; incident queued with policy_degraded = true.
3. Alert Circadia ops; banner on Command and manager desk.
4. Never drop inbound alerts.

True fail-safe for life-safety: never break edge ingest bridge.

Part G - Subscription layer

Tenant

id, name, tenantIdUuid

isolationTier: shared | branch | project

stripeCustomerId, subscriptionId, planId

status: trialing | active | past_due | cancelled

entitlements: commandDesk, liveAlerts, maxDrivers, maxOperators, routingMode

Provisioning webhook

checkout.session.completed -> create Tenant, tenantIdUuid, routing from sales worksheet

subscription.updated -> enable/disable bridge; suspend ingest if cancelled

Gating: no active sub -> bridge off; assurance-only plan -> pipeline B only; Command + Manager -> M1/M2 + operator assignment.

Part H - Revised phase delivery sequence

Merged from original outline + design review. Identity sync moved up - do not onboard second paying customer on heuristic hashes alone.

Phase | Scope

|-----|-----|

- 1 | Tenant entity + User.tenantId + Driver.tenantId + per-tenant settings (replace singletons)
 - 2 | Identity sync worker + identity_uuid_map + provisional/unmapped states + manager reconciliation UI
 - 3 | Option A isolation: selective RLS, operator assignment tier 1, tenant-scoped queues/SSE, remove pilot env UUID
 - 4 | Subscription + entitlements + onboarding wizard (M1-M4 choice)
 - 5 | Isolation tier upgrades via database routing service (Neon branch/project)
 - 6 | Operator tier-2 SLA cascade + advanced desk ops (optional)
-

Part I - Gap summary

Gap | Impact

|----|-----|

No Tenant in app-next | Cannot assign users, drivers, settings per customer
 No operator-tenant assignment | All operators see all incidents
 Global Command queue | Multi-tenant ingest mixes queues
 RLS global bypass | No DB-enforced tenant boundary
 Single pilot env UUID | Bridge hard-coded to one customer
 No subscription entitlements | Cannot gate Command by payment
 No identity sync worker | Driver UUID mapping incomplete
 routing_mode not in DB | Per-tenant assembly not persistable
 Prisma single DATABASE_URL | Blocks Option B/C without routing service

Part J - Architecture review verdict

Recommendation | Verdict

|-----|-----|

Hybrid DB router | Adopt - core enabler for B/C without forked apps
 RLS on all tables | Partial - selective RLS + service ingest path
 Composite keys / portability | Adopt - plan early on event tables
 Unmapped ingest catchment | Adopt - provisional/unmapped status, not null UUID
 Manager reconciliation UI | Adopt - fits /manager/alerts
 Tiered operator assignments | Adopt schema - Tier 2 cascade in Phase 6
 Routing as state machine | Adopt - mandatory before multi-tenant go-live
 Fail-safe to M3 | Revise - degrade visibly, do not silently reroute

Move identity sync up | Adopt - Phase 2, immediately after Tenant entity

Hard rule (unchanged): Do not merge Command into app-next. Managers get bridge APIs only (MASTER_SPEC section 1).

Part K - Decisions to lock before build

1. Default isolation tier - shared + RLS for SMB; branch/project for enterprise only?
 2. Billing customer - fleet owner in app-next, or Circadia bills and provisions tenant?
 3. Command desk model - one Circadia desk serving many tenants vs dedicated URL per large customer?
 4. Routing at sale - force one of M1-M4 at onboarding?
 5. Data residency - AU Neon today; per-tenant region only for Option C?
 6. Fail-safe policy - cached last-known-good vs explicit ops alert threshold?
-

Part L - Related documents

Document | Path

|-----|-----|

Incident routing assembly | app-next/docs/architecture/incident-routing-assembly.md

Command MASTER_SPEC | circadia-command/docs/MASTER_SPEC.md

Schema adaptations | circadia-command/docs/SCHEMA_ADAPTATIONS.md

Deploy / env vars | circadia-command/docs/DEPLOY_VERCEL.md

Manager critical alerts | app-next/docs/architecture/manager-critical-alert-spec.md

ADR Postgres scaling | app-next/docs/adr/0002-managed-postgres-and-data-access.md

Circadia 24 - Confidential project outline v2 - Internal planning